X.509 Certificate Generation Code Explanation

This C program generates a self-signed X.509 certificate using OpenSSL. Let me break down each section:

Headers and Libraries

```c
#include <openssl/x509.h>    // X.509 certificate structures and functions
#include <openssl/pem.h>     // PEM format encoding/decoding
#include <openssl/rsa.h>     // RSA key generation and operations
#include <openssl/pkey.h>    // Generic public/private key operations
#include <openssl/evp.h>     // High-level cryptographic functions
#include <openssl/bn.h>      // Big number arithmetic
#include <stdio.h>           // Standard I/O operations
```

Step 1: RSA Key Generation

```c
EVP_PKEY *pkey = EVP_PKEY_new();
RSA *rsa = RSA_new();
BIGNUM *bn = BN_new();
BN_set_word(bn, RSA_F4);
RSA_generate_key_ex(rsa, 2048, bn, NULL);
EVP_PKEY_assign_RSA(pkey, rsa);
BN_free(bn);
```

Variables and Functions:

- `EVP_PKEY *pkey`: Generic key container that can hold different key types (RSA, DSA, etc.)
- `RSA *rsa`: Specific RSA key structure
- `BIGNUM *bn`: Big number for storing the public exponent
- `EVP_PKEY_new()`: Creates a new empty key container
- `RSA_new()`: Creates a new RSA key structure
- `BN_new()`: Creates a new big number
- `BN_set_word(bn, RSA_F4)`: Sets the public exponent to 65537 (RSA_F4 = 0x10001)
- `RSA_generate_key_ex()`: Generates a 2048-bit RSA key pair with the specified exponent
- `EVP_PKEY_assign_RSA()`: Transfers ownership of the RSA key to the EVP_PKEY container

Step 2: Certificate Creation

```c
X509 *x509 = X509_new();
```

Variables:

- `X509 *x509`: Structure representing an X.509 certificate

Step 3: Version and Serial Number

```c
ASN1_INTEGER_set(X509_get_serialNumber(x509), 1);
X509_set_version(x509, 2);
```

Functions:

- `X509_get_serialNumber()`: Gets the certificate's serial number field
- `ASN1_INTEGER_set()`: Sets an ASN.1 integer value (serial number = 1)
- `X509_set_version()`: Sets certificate version (2 = v3, which is the current standard)

Step 4: Validity Period

```c
X509_gmtime_adj(X509_get_notBefore(x509), 0);
X509_gmtime_adj(X509_get_notAfter(x509), 60*60*24*365);
```

Functions:

- `X509_get_notBefore()`: Gets the "valid from" timestamp field
- `X509_get_notAfter()`: Gets the "valid until" timestamp field
- `X509_gmtime_adj()`: Adjusts time relative to current GMT
  - First call: Sets start time to now (0 seconds offset)
  - Second call: Sets end time to 365 days from now (60×60×24×365 seconds)

Step 5: Subject and Issuer Names

```c
```

```
X509_NAME *name = X509_get_subject_name(x509);
X509_NAME_add_entry_by_txt(name, "C", MBSTRING_ASC,
                           (unsigned char *)"US", -1, -1, 0);
X509_NAME_add_entry_by_txt(name, "O", MBSTRING_ASC,
                           (unsigned char *)"MyOrg", -1, -1, 0);
X509_NAME_add_entry_by_txt(name, "CN", MBSTRING_ASC,
                           (unsigned char *)"example.com", -1, -1, 0);
X509_set_issuer_name(x509, name);
```

Variables and Functions:

- `X509_NAME *name`: Structure containing certificate name information
- `X509_get_subject_name()`: Gets the certificate subject (who the cert identifies)
- `X509_NAME_add_entry_by_txt()`: Adds name components:
  - "C": Country (US)
  - "O": Organization (MyOrg)
  - "CN": Common Name (example.com - the domain name)
  - `MBSTRING_ASC`: Specifies ASCII string encoding
  - `-1, -1, 0`: Location and set parameters (defaults)
- `X509_set_issuer_name()`: Sets who signed the certificate (same as subject = self-signed)

Step 6: Public Key Assignment

```
c

X509_set_pubkey(x509, pkey);
```

Function:

- `X509_set_pubkey()`: Associates the public key with the certificate

Step 7: Digital Signature

```
c

X509_sign(x509, pkey, EVP_sha256());
```

Function:

- `X509_sign()`: Signs the certificate with the private key using SHA-256 hash algorithm

- This creates the digital signature that proves the certificate's authenticity

Step 8: File Output

```c
FILE *f = fopen("cert.pem", "wb");
PEM_write_X509(f, x509);
fclose(f);

f = fopen("key.pem", "wb");
PEM_write_PrivateKey(f, pkey, NULL, NULL, 0, NULL, NULL);
fclose(f);
```

Functions:

- `PEM_write_X509()`: Writes certificate in PEM format (Base64-encoded, human-readable)
- `PEM_write_PrivateKey()`: Writes private key in PEM format
  - NULL parameters mean no encryption/password protection

Step 9: Memory Cleanup

```c
X509_free(x509);
EVP_PKEY_free(pkey);
```

Functions:

- `X509_free()`: Frees certificate memory
- `EVP_PKEY_free()`: Frees key memory (also frees the RSA key since it was assigned)

Key Concepts

X.509 Certificate: A digital certificate standard that binds a public key to an identity (like a domain name). Contains:

- Public key
- Identity information (subject name)
- Digital signature from a trusted authority
- Validity period

- Serial number

Self-Signed Certificate: The certificate is signed by its own private key rather than a Certificate Authority (CA). Useful for testing but browsers will show security warnings.

PEM Format: Privacy-Enhanced Mail format - a Base64 encoding of certificate data with "BEGIN CERTIFICATE" headers, making it human-readable.

RSA Key Pair: Asymmetric encryption with public key (shared) and private key (kept secret). The 2048-bit key size provides good security.